

vasic-digital/tmux — v1.0.22 (versionCode 23) — 2026-06-13

Revision: 1 Last modified: 2026-06-13T16:30:00Z

Apple container integration: on-demand containerized Linux under macOS for testing — build tmux 3.6a inside a real Linux VM and run the suite, no remote Linux host required.

What shipped

This release adds a host-local way to build and test the **Linux** build of tmux from a **macOS** workstation, using Apple's native container runtime (1.0.0). It is a *testing-capability* release — the shipped tmux 3.6a binary and the tmx wrapper are byte-for-byte identical to v1.0.21; no product behaviour changed.

Two pieces:

1. **Containers-submodule backend** — Apple container 1.0.0 was incorporated into the vasic-digital/Containers submodule as a new generic pkg/crossbuild/apple_container.go backend exposing RunInLinuxContainer, with unit tests, an integration test, and a challenge. This is reusable by any consuming project, not tmux-specific.
2. **tmx-side harness** — scripts/test_apple_container.sh builds tmux 3.6a inside an Apple-container Linux VM and runs the project's own run_all.sh suite against that Linux binary, capturing real PASS/FAIL/SKIP evidence.

Why (motivation)

The project is native dual-OS (runs as a host process on Linux **and** macOS). But on an Apple-Silicon macOS workstation there was previously no host-local way to exercise the **Linux** build — Linux validation depended on the remote nezha host being reachable. This release gives on-demand Linux-under-macOS so a developer can build the genuine Linux ELF tmux binary and run the suite with one command, on their own machine, with no remote host.

Requirements

- macOS 15+ on Apple Silicon.
- Apple container 1.0.0 or newer, installed and running:
 - brew install container

- container system start (the harness verifies container system status is running and that a Linux kernel image is present).
- Network access to pull the base image (docker.io/library/ubuntu:22.04 by default). On a macOS-behind-VPN host with no outbound network inside the VM, the harness downloads apt .deb build-deps host-side and injects them.

How to use

1. Ensure the Apple container runtime is running:

container system start

2. Build tmux 3.6a inside a Linux VM and run the suite against it:

bash scripts/test_apple_container.sh

Options:

- `--image IMG` (or `TMX_AC_IMAGE`) — base Linux image (default docker.io/library/ubuntu:22.04).
- `--keep` — leave the transient container in place for debugging.
- `--no-build` — skip the build (expects a prior `--keep` run's binary).

Honest exit codes:

Code	Meaning
0	suite ran and reported PASS (FAIL=0) — real success
1	suite ran and reported ≥ 1 FAIL — real product defect
2	build failed inside the container — toolchain/source defect
3	SKIP — Apple container runtime / kernel absent, or not macOS (safe no-op)

The harness stops and removes the transient container on **every** exit path (unless `--keep`).

Proven evidence

Captured this cycle on a macOS 15.5 / arm64 host, under docs/qa/2026-06-13-apple-container/linux-run/:

Artifact	Proof
uname.txt	in-container <code>uname -s -m</code> = Linux aarch64 (host is Darwin/arm64 — a genuine Linux VM)
tmux-version.txt	built binary reports tmux 3.6a
elf-proof.txt	ELF magic 7f 45 4c 46 + <code>ldd</code> shows <code>libjemalloc.so.2</code> , <code>libtinfo</code> , <code>libevent_core</code> , <code>libc</code> — a real Linux ELF, jemalloc-linked
build.log	

Artifact	Proof
run_all.log / summary.txt	full in-VM configure + compile transcript (Linux gcc, osdep-linux.o) suite result PASS=30 / FAIL=0 / SKIP=28 against the Linux binary

Run context:

- **Host:** Darwin arm64
- **Container OS:** Linux aarch64
- **Base image:** docker.io/library/ubuntu:22.04
- **Built binary:** tmux 3.6a (jemalloc-mapped, verified via /proc/<pid>/maps)

Containers-submodule backend (proven separately):

- Real container run returns Linux aarch64 on this Darwin host.
- Host-directory mount round-trip works.
- Paired mutation that strips the --mount flag exits 99 (mount contract mechanically enforced).

Honest limitations / SKIPS

The 28 SKIPS are honest topology gaps (§11.4.3 / §11.4.81) — never silent passes, never failures. The minimal container VM intentionally lacks several host capabilities:

- **No user systemd session** → cgroup/scope tests SKIP-with-reason: 08_oom_score_adj, 09_crash_isolation_scope, 12_memory_pressure_under_cap, 13_tasksmax_stress, 14_concurrent_oom_independence, 15_per_session_cgroup_distinct, 24_cpu_cap_enforcement.
- **No DISPLAY / PTY tty** → physical-terminal, real-clipboard, real-mouse tests SKIP: 44–48, 55–59.
- **No host tooling / cross-host context** → SKIP: 16_window_name_strips_exe, 18_constitution_inheritance, 20–22 (CodeGraph), 32_ssh_dispatch_remote_nezha, 39_state_unwritable, 40_macos_linux_parity, 41_docs_user_guides_render, 43_e2e_cwd_persist_real_shell, 51_workable_items_db_integrity.

The 30 core tmux tests — smoke, session lifecycle, jemalloc-mapped proof, history-limit respect, clear-history, scrollbar, hostname-colour, and the rest of the Linux-relevant set — RAN and PASSED. For the full cgroup/systemd isolation suite, use the dedicated Linux host (nezha); this harness covers the core tmux behaviour under Linux on a macOS workstation.

Constitution compliance

- **§11.4.76 (Containers-submodule mandate):** the containerization capability was added to the vasic-digital/Containers submodule and consumed from there — not reimplemented in this project.

- **§11.4.74 (catalogue-first, extend-don't-reimplement):** the Apple container backend extends the existing submodule rather than duplicating an orchestration layer in-tree.
- **§11.4.3 / §11.4.81 (topology dispatch + cross-platform parity):** absent capabilities SKIP-with-reason; the harness never PASS-by-defaults.
- **§11.4.6 (no-guessing):** every claim above is backed by a captured artefact in docs/qa/2026-06-13-apple-container/linux-run/.

Links

- Changelog entry: CHANGELOG.md → ## [v1.0.22] – 2026-06-13
- Closed-item record: Fixed.md → A44 (Type: Feature, Implemented)
- Harness: scripts/test_apple_container.sh
- Design notes: docs/qa/2026-06-13-apple-container/DESIGN.md
- Evidence: docs/qa/2026-06-13-apple-container/linux-run/

Sources verified 2026-06-13

- Apple container documentation — <https://apple.github.io/container/documentation/>